

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе № 3

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

Тема: «Динамические структуры данных (списки, стек, очередь, приоритетная очередь)»

Выполнили:

студенты гр. 25BBB1:

Ячменев М. А.

Аверин А. А.

Приняли:

к.т.н., доцент Юрова О. В.

ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение принципов организации и функционирования динамических структур данных (односвязные линейные списки, стек, очередь, приоритетная очередь). Приобретение практических навыков динамического управления оперативной памятью в языке программирования Си с использованием функций `malloc()` и `free()`, разработки алгоритмов добавления, извлечения, сквозного просмотра, поиска и удаления элементов в связанных структурах данных.

2. Лабораторное задание

В соответствии с методическими указаниями к лабораторной работе № 3 требуется выполнить следующие задания:

- 1. Изучить базовую реализацию односвязного линейного списка, содержащего текстовые наименования объектов (`inf[256]`), функции добавления (`spstore`), просмотра (`review`), поиска (`find`) и удаления (`del`).
- 2. На основе базовой связной структуры реализовать структуру данных Очередь (Queue), работающую по принципу FIFO (First In, First Out — первым пришел, первым обслужен), с операциями постановки в хвост и извлечения из головы.
- 3. На основе приведенного связного списка реализовать структуру данных Стек (Stack), функционирующую по принципу LIFO (Last In, First Out — последним пришел, первым обслужен), с операциями помещения в вершину (`push`) и извлечения из вершины (`pop`).
- 4. Реализовать структуру данных Приоритетная очередь (Priority Queue) путем упорядоченного включения элементов в список в соответствии с числовым приоритетом объекта (элемент с более высоким приоритетом размещается перед элементом с меньшим приоритетом, а при равных приоритетах сохраняется естественный порядок поступления FIFO).
- 5. Реализовать функции поиска и выборочного удаления элементов по наименованию в приоритетной очереди, а также функции полной очистки выделенной динамической памяти.
- 6. Разработать демонстрационный модуль и интерактивное меню пользователя в среде Microsoft Visual Studio на языке Си, провести тестирование всех реализованных структур и составить подробный отчет о результатах работы.

3. Теоретические сведения

3.1. Особенности динамических структур данных и управление памятью

В отличие от статических структур данных (таких как стандартные одномерные и многомерные массивы), размер которых фиксируется на этапе компиляции и не может изменяться в процессе выполнения программы, динамические структуры данных обладают следующими ключевыми свойствами:

- Динамическое выделение памяти: память под каждый элемент (узел) выделяется из свободной области оперативной памяти (кучи, heap) непосредственно в момент создания элемента по мере необходимости.
- Связность через указатели: узлы структуры не обязаны располагаться в смежных ячейках оперативной памяти; логическая связь между элементами обеспечивается явным хранением адреса следующего узла в поле-указателе.
- Отсутствие жесткого ограничения на размер: структура может неограниченно расти вплоть до полного исчерпания физической оперативной памяти вычислительной системы и легко уменьшаться при освобождении узлов.

В языке Си управление динамической памятью осуществляется посредством стандартных функций библиотеки `<stdlib.h>`:

- `malloc(size)` — выделяет непрерывный блок памяти размером `size` байт и возвращает нетипизированный указатель (`void*`) на его начало, либо `NULL` при нехватке памяти.
- `free(ptr)` — освобождает ранее выделенный блок памяти по адресу `ptr`, возвращая его операционной системе для повторного использования.

Базовым строительным блоком динамического списка является узел (структура `node`), определяемый следующим образом:

```
struct node {  
    char inf[256];    // полезная информация (наименование объекта)  
    int priority;     // приоритет элемента (для приоритетной очереди)  
    struct node *next; // указатель на следующий элемент списка  
};
```

3.2. Принцип функционирования Стекa (LIFO)

Стек (Stack) — динамическая структура данных, работающая по принципу LIFO (Last In, First Out — «последним пришел, первым обслужен»). Доступ к

элементам осуществляется исключительно через один конец структуры, называемый вершиной стека (TOP):

- Операция push (проталкивание): новый элемент связывается с текущей вершиной, после чего указатель вершины смещается на вновь добавленный узел. Сложность операции — $O(1)$.
- Операция pop (выталкивание): из вершины извлекается информационное поле, указатель вершины переносится на следующий элемент (next), а память прежней вершины освобождается вызовом free(). Сложность операции — $O(1)$.

3.3. Принцип функционирования Очереди (FIFO)

Очередь (Queue) — структура данных, организованная по дисциплине обслуживания FIFO (First In, First Out — «первым пришел, первым ушел»). Для эффективного функционирования используются два независимых указателя:

- Указатель на начало (HEAD): через него осуществляется извлечение обслуженных элементов (dequeue).
- Указатель на конец (TAIL): через него осуществляется добавление новых элементов в хвост очереди (enqueue).

Использование двух указателей обеспечивает константную временную сложность $O(1)$ как для операции добавления в хвост, так и для извлечения из головы.

3.4. Принцип функционирования Приоритетной очереди

Приоритетная очередь (Priority Queue) — структура данных, в которой каждый элемент помимо данных несет характеристику важности (приоритет priority). Обслуживание элементов производится строго в порядке убывания их приоритета (наивысший приоритет обслуживается первым):

- Алгоритм упорядоченной вставки (enqueue): при поступлении нового элемента список просматривается от головы; элемент размещается перед первым встреченным узлом, имеющим строго меньший приоритет.
- Стабильность очереди (FIFO для равных приоритетов): если в очереди уже присутствуют элементы с таким же приоритетом, новый элемент помещается за ними (условие $\text{next} \rightarrow \text{priority} \geq \text{priority}$), что гарантирует справедливую очередность обслуживания.
- Извлечение максимального элемента (dequeue): поскольку список постоянно поддерживается в отсортированном по невозрастанию приоритетов

состоянии, элемент с наивысшим приоритетом всегда находится в голове очереди (HEAD), что позволяет извлечь его за время $O(1)$.

1. Стек (Stack — LIFO: Last In, First Out)



2. Очередь (Queue — FIFO: First In, First Out)



3. Приоритетная очередь (Priority Queue — упорядочение по приоритету)



* При одинаковом значении приоритета новый элемент ставится ПОСЛЕ существующего (сохранение FIFO)

Рисунок 1 – Схемы организации структур данных: Стек (LIFO), Очередь (FIFO) и Приоритетная очередь

4. Описание программной реализации

Программный комплекс разработан в среде Microsoft Visual Studio на стандартном языке Си (C99/C11). Проект структурирован в виде консольного приложения и включает следующие компоненты:

- alg3lr.sln / alg3lr.slnx — файлы решений Visual Studio для удобного открытия проекта в один клик в среде разработки.
- alg3lr.vcxproj — файл конфигурации проекта C/C++ с параметрами сборки под платформы x86 и x64.
- alg3lr.c — основной файл исходного кода, содержащий полную реализацию структур данных, демонстрационного тестирования и пользовательского диалогового меню.

Программа состоит из следующих функциональных блоков:

- Модуль стека: функции `stack_push()` — выделение узла и включение в вершину `stack_top`; `stack_pop()` — удаление вершины с возвратом данных; `stack_review()` — итеративный вывод элементов; `stack_clear()` — полное циклическое освобождение памяти.
- Модуль очереди: функции `queue_enqueue()` — добавление в `queue_tail` с обновлением связей; `queue_dequeue()` — извлечение из `queue_head`; `queue_review()` — просмотр от головы к хвосту; `queue_clear()` — очистка всех узлов очереди.
- Модуль приоритетной очереди: функции `pq_enqueue()` — однопроходный линейный поиск позиции вставки по убыванию поля `priority`; `pq_dequeue()` — немедленное извлечение элемента из головы списка `pq_head`; `pq_find()` — последовательный поиск элемента по строковому наименованию `strcmp()`; `pq_del()` — перелинковка указателей и освобождение найденного узла; `pq_review()` и `pq_clear()`.
- Диалоговый интерфейс: функция `main()` реализует текстовое консольное меню с поддержкой кодовой страницы Windows-1251 для корректного отображения кириллических символов, обработкой ошибок ввода и пунктом 1 для запуска автоматического демонстрационного теста.

Таблица 1 – Сравнительная характеристика реализованных структур данных

Структура данных	Принцип доступа	Сложность вставки	Сложность извлечения	Точка добавления	Точка извлечения
Стек (Stack)	LIFO	O(1)	O(1)	Вершина (TOP)	Вершина (TOP)
Очередь (Queue)	FIFO	O(1)	O(1)	Хвост (TAIL)	Голова (HEAD)
Приоритетная очередь	По приоритету (Priority)	O(N)	O(1)	По позиции приоритета	Голова (HEAD)
Линейный список	Произвольный	O(1) в конец	O(N) по ключу	Конец (last)	По значению (del)

5. Анализ результатов тестирования:

1. Корректность работы Стекa: при добавлении документов 1, 2, 3 последним был добавлен «Документ_3». При выполнении операции `pop()`

именно он был первым извлечен из вершины, что полностью подтверждает дисциплину LIFO.

2. Корректность работы Очереди: элементы «Клиент_А», «Клиент_Б», «Клиент_В» добавлялись в хвост очереди. При вызове dequeue() первым был обслужен «Клиент_А», первым вставший в очередь, что доказывает соблюдение принципа FIFO.

3. Корректность работы Приоритетной очереди: при добавлении элементов с различными приоритетами (от 1 до 10) структура автоматически выстроила элементы строго по убыванию приоритета. «Критический_сбой_системы» (приоритет 10) занял первое место в очереди, опередив ранее добавленные элементы с меньшим приоритетом.

4. Сохранение очередности при равных приоритетах: элементы «Запрос_базы_данных_1» и «Запрос_базы_данных_2» имеют одинаковый приоритет 5. В очереди «Запрос_базы_данных_1» остался перед «Запросом_базы_данных_2», сохранив хронологический порядок добавления.

5. Поиск и удаление: элемент «Отправка_логов» был успешно найден по наименованию и корректно удален из середины связного списка с восстановлением целостности указателей.

6. Выводы

- Изучены теоретические основы организации динамических структур данных на базе связных списков и указателей в языке программирования Си.
- Освоены практические навыки динамического выделения (malloc) и своевременного освобождения (free) оперативной памяти, исключающие утечки памяти.
- На основе базового односвязного списка успешно реализованы классические структуры данных Стек (LIFO) и Очередь (FIFO).
- Реализована Приоритетная очередь с автоматической упорядоченной вставкой элементов по убыванию приоритета и сохранением хронологического порядка (FIFO) для элементов с одинаковым приоритетом.
- Реализованы функции поиска и удаления произвольных узлов списка по строковому наименованию, а также полное освобождение динамических структур при завершении программы.
- Проведено всестороннее тестирование разработанных алгоритмов в среде Microsoft Visual Studio, полностью подтвердившее корректность их функционирования.